

2 Computation Graphs

2.1 Logistic Regression

We've already got to know the maximum likelihood estimator and more precisely the Conditional **Maximum Likelihood Estimator** for $\mathbf{w}$:

$$\hat{\mathbf{w}}_{ML} = \underset{\mathbf{w}}{\operatorname{argmax}} \sum_{i=1}^N \log p_{model}(y_i | \mathbf{x}_i, \mathbf{w}) \quad (1)$$

because we're conditioning on some input $\mathbf{x}$ we're not just trying to model a distribution y but we are trying to model a conditional distribution y given $\mathbf{x}$. We can see here the Maximum Loglikelihood estimator, we have the log of the product, which is the sum of the logarithm. However, the estimate that we obtain - argmax , is the same as the Maximum Likelihood Estimator would return as the logarithm is a monotonic function. In practice we prefer to use loglikelihood estimator both for numerical reasons when implementing these algorithms because this is resulting in more stable computation, but also for mathematical reasons and for connection to the concepts in information theory. We'll mostly use loglikelihood from now, it doesn't return the same value, but it returns the same maximum, the same parameters $\mathbf{w}$. We denote the estimate as $\hat{\mathbf{w}}_{ML}$. This hat indicates that this is an estimate of the true parameters. Now we want to perform a binary classification: $y_i \in \{0, 1\}$. So the output can take only two possible discrete labels - 0 and 1. The question is - how should we choose $p_{model}(y | \mathbf{x}, \mathbf{w})$ in this case? Gaussian model isn't a good choice, because it's a continuous distribution. Bernoulli distribution is good for this case, it models binary classification problem:

$$p_{model}(y | \mathbf{x}, \mathbf{w}) = \hat{y}^y (1 - \hat{y})^{(1-y)} \quad (2)$$

where $\hat{y}$ is a prediction of some model and y is the true label from our dataset. So $\hat{y}$ is some prediction that must depend on the input $\mathbf{x}$ and it must also depend on some trainable parameter $\mathbf{w}$, we denote that function as: $\hat{y} = f_{\mathbf{w}}(\mathbf{x})$.

We are working with the discrete distribution, so we have the next requirement $f_{\mathbf{w}}(\mathbf{x}) \in [0, 1]$. We can use a sigmoid function: $f_{\mathbf{w}}(\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x})$ where σ is the sigmoid function:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (3)$$

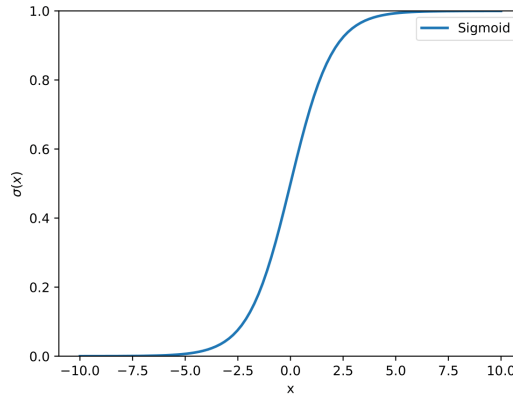


Figure 4: **Sigmoid.** An illustration of the Sigmoid function.

We take the linear combination of the weights and then we do the non-linear transformation sigma to it. Sigmoid takes an unbounded range of the real numbers and transform this domain of real numbers to the domain of from 0 to 1 interval. That's why this function is called squashing function. And that's what we want - we want the output of this function between 0 and 1 to be a proper probability. That's also a reason why is this called a logistic regression, because this is logistic transfer function.

Lets put it together:

$$\hat{\mathbf{w}}_{ML} = \underset{\mathbf{w}}{\operatorname{argmax}} \sum_{i=1}^N \log p_{model}(y_i | \mathbf{x}_i, \mathbf{w}) \quad (4)$$

$$= \underset{\mathbf{w}}{\operatorname{argmax}} \sum_{i=1}^N \log \left[\hat{y}_i^{y_i} (1 - \hat{y}_i)^{(1-y_i)} \right] \quad (5)$$

$$= \underset{\mathbf{w}}{\operatorname{argmin}} \sum_{i=1}^N \underbrace{-y_i \log \hat{y}_i - (1 - y_i) \log(1 - \hat{y}_i)}_{\text{Binary Cross Entropy Loss } \mathcal{L}(\hat{y}_i, y_i)} \quad (6)$$

Here for the model distribution we plug in the Bernoulli distribution, where

$$\hat{y}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$$

. Because we have a logarithm, we can transform it, we split the product with a sum. And then instead of maximizing we can minimize the last expression by writing a minus in front. In machine learning we often don't maximize likelihoods, but we rather minimize loss functions and that's why we are writing it this way here and this term in the sum has a specific name in the machine learning community, it is called a **binary cross-entropy loss**. It's a loss between the predicted $\hat{y}$ and the true y and as the name loss indicates a high loss is bad and a low loss is desirable, so we want to minimize the loss, we want to compute the overall minimum over the data set of all these individual losses. In machine learning we use often the more general term loss rather than what we have used before in linear regression the error function and the reason is that this loss term is more general, it's not necessarily an error between observations, it could also be for instance an inductive bias that we want to encode. Now this formula has the following interpretation, we minimize the dissimilarity between the empirical data distribution p_{data} (defined by the training set) and the model distribution p_{model} . We want to find a parameter for our model, such that the prediction of our model is most similar to our data. Lets look at the Binary cross entropy loss:

$$\mathcal{L}(\hat{y}_i, y_i) = -y_i \log \hat{y}_i - (1 - y_i) \log(1 - \hat{y}_i) \quad (7)$$

$$= \begin{cases} -\log \hat{y}_i & \text{if } y_i = 1 \\ -\log(1 - \hat{y}_i) & \text{if } y_i = 0 \end{cases} \quad (8)$$

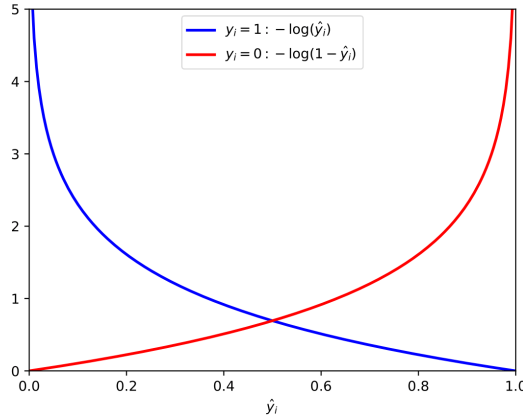


Figure 5: **Curves.** An illustration of the those two curves.

We want to minimize those curves. For $y_i = 1$ the loss $\mathcal{L}$ is minimized if $\hat{y}_i = 1$. For $y_i = 0$ the loss $\mathcal{L}$ is minimized if $\hat{y}_i = 0$. Thus, $\mathcal{L}$ is minimal if $\hat{y}_i = y_i$. This loss function can be extended to > 2 classes.

2.1.1 1D example

Lets look at this visually.

We have on the x-axis our features, this is the input 1d and we want to classify between positive and negative. So we want to tell for a particular x value is it in the positive class or in a negative class. We have set of positive samples in green and negative in red. We want to fit to dataset $\mathcal{X}$ logistic regression model $f_{\mathbf{w}}(x) = \sigma(w_0 + w_1 x)$

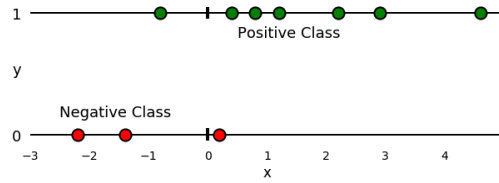


Figure 6: **1D example.** An illustration of the simple 1D example.

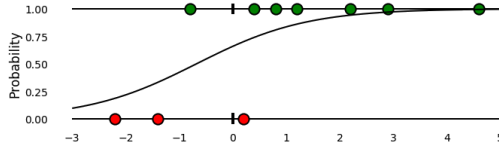


Figure 7: **Model fitting.** An illustration of fitting the log regression model.

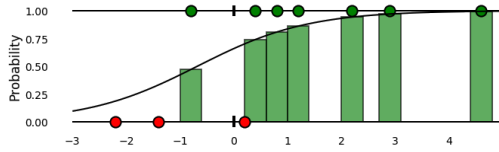


Figure 8: **Probabilistic interpretation.** Probabilities of classifier $f_{\mathbf{w}}(x_i)$ for positive samples ($y_i = 1$)

How do we interpret this in terms of probabilities and the loss function?

The probabilities of the classifier for the positive samples are shown with green bars here where each of these green bars is basically extending all the way up to the curve because this is the probability that the model assigns to these positive values. What we can see is already the criterion that we want to optimize, for all data points we want to maximize these probabilities being correctly assigned.

And these are the probabilities for the negative points Now we can put all of these probabilities together and

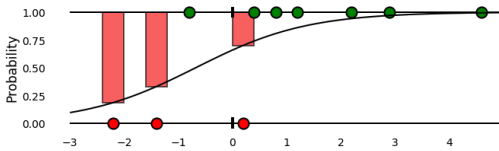


Figure 9: **Probability of the negative class.** Probabilities of classifier $f_{\mathbf{w}}(x_i)$ for negative samples ($y_i = 0$)

we'll just put them to the bottom, because it makes more sense.

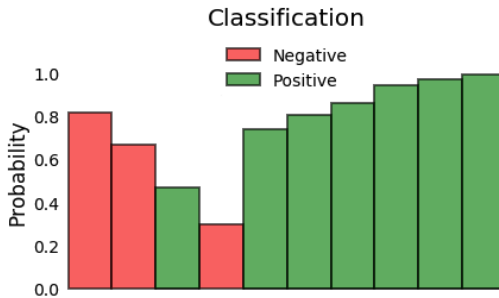


Figure 10: **Probability of the both classes.** Probabilities of classifier for the both classes

We want to maximize those probabilities or equivalently we want to minimize the negative logarithm. So then we minimize the mean or the sum over all these negative log probabilities.

2.1.2 Optimizing $\mathbf{w}$.

In contrast to linear regression, the loss $\mathcal{L}(\hat{y}_i, y_i)$ is **not quadratic** in $\mathbf{w}$. $\mathbf{w}$ appears inside the sigmoid which is a nonlinear function and then we take the logarithm of this nonlinear function, which is also a non-linear

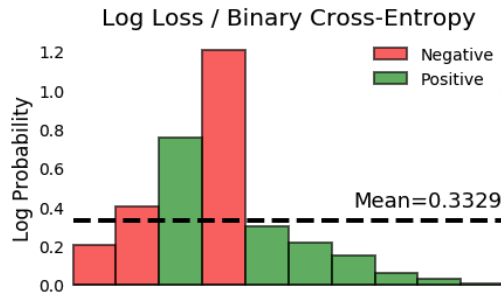


Figure 11: **Log loss.** Log loss of classifier for both classes

function, so this is highly non-linear transformation of linear model and we don't have an easy expression as we had it in the quadratic case for the linear regression task. If there is no closed form analytic solution, we need to apply some iterative optimization technique which is based on gradients. Luckily we can compute gradients and so we can use a gradient based optimizer, that slowly follows the gradient to a local minimum starting from some initialization point. Despite this not being a quadratic function we, it's still a convex function. It can be shown that this problem is a convex problem, so we know that we don't fall into a local optimum. If we apply a gradient based optimizer, we end up with the global optimum. But we can't reach it in a single step or analytically, we need to apply an iterative gradient-based algorithm. And for all the gradient-based optimization algorithms, where the grading is needed, we need to compute this gradient which is the nabla operator. The nice thing about this particular objective function is that despite in the in the first step the gradient looks quite complex, it simplifies a lot when you do the analytical derivation and this is the final form of the gradient of the binary cross entropy loss of this logistic regression model:

$$\nabla_{\mathbf{w}} \mathcal{L}(\hat{y}_i, y_i) = (\hat{y}_i - y_i) \mathbf{x}_i \quad (9)$$

Since we are able to compute this gradient analytically, we can apply it inside a iterative gradient based optimizer, which tries to go stepwise towards the optimum. The simplest optimizer that we can use and one that's actually used nowadays heavily even in deep learning is called gradient descent. We pick the step size η and tolerance ϵ and Initialize $\mathbf{w}^0$. And then we repeat until $\|\mathbf{v}\| < \epsilon$. We iterate the procedure:

$$\mathbf{v} = \nabla_{\mathbf{w}} \mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_{i=1}^N \nabla_{\mathbf{w}} \mathcal{L}(\hat{y}_i, y_i) \quad (10)$$

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \mathbf{v} \quad (11)$$

We optimize this until this gradient becomes small and converges to an extreme value of this function.

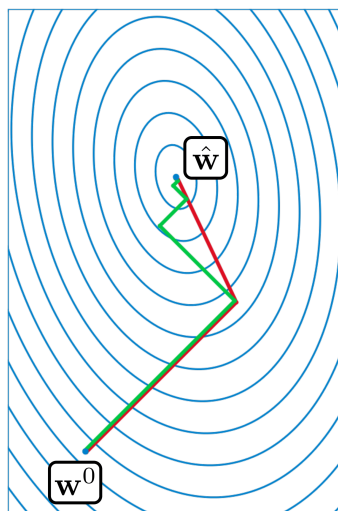


Figure 12: **Gradient descent.** Example of the converging variants of gradient descent.

But there's several variants that work a little bit better, one is line search, that's the green curve, where you're going into the direction of the gradient, but you search for the smallest value. You can do even better for some problems by using the conjugate gradient method (red), where you're not going into the direction of the gradient, but into some conjugate direction, that brings you more directly to the optimal value.

2.1.3 Connections to the Information Theory

Maximizing the **Log-Likelihood** is equivalent to minimizing **Cross Entropy** or **KL Divergence**:

$$\hat{\mathbf{w}}_{ML} = \underset{\mathbf{w}}{\operatorname{argmax}} \underbrace{\sum_{i=1}^N \log p_{model}(y_i | \mathbf{x}_i, \mathbf{w})}_{\text{Log-Likelihood}} \quad (12)$$

$$= \underset{\mathbf{w}}{\operatorname{argmax}} \mathbb{E}_{p_{data}} [\log p_{model}(y | \mathbf{x}, \mathbf{w})] \quad (13)$$

$$= \underset{\mathbf{w}}{\operatorname{argmin}} \underbrace{-\mathbb{E}_{p_{data}} [\log p_{model}(y | \mathbf{x}, \mathbf{w})]}_{\text{Cross Entropy } H(p_{data}, p_{model})} \quad (14)$$

$$= \underset{\mathbf{w}}{\operatorname{argmin}} \mathbb{E}_{p_{data}} [\log p_{data}(y | \mathbf{x}) - \log p_{model}(y | \mathbf{x}, \mathbf{w})] \quad (15)$$

$$= \underset{\mathbf{w}}{\operatorname{argmin}} \underbrace{D_{KL}(p_{data} || p_{model})}_{\text{KL Divergence}} \quad (16)$$

We can rewrite log likelihood using an expectation, because what if we would write $1/N$ over N , which would be the mean, then we would look at the empirical estimate over the data distribution, but $1/N$ is a constant with respect to $\mathbf{w}$ so it gets absorbed into the argmax operator. But equation having a $1/N$ in the beginning would exactly correspond to the definition of the expectation operator, which is basically expectation over the probability of the data points. So each of the data points is a sampling based estimate of the $\log p_{model}$. We can write argmax in terms of argmin , if we replace plus with a minus. Now what we can do also is we can take this term and add the $\log p_{data}(y | \mathbf{x})$. We can do that because the data probability in this data distribution does not depend on $\mathbf{w}$, so it's changing the value of this function, but it's constant with respect to $\mathbf{w}$. It means that we have the cross-entropy minus the entropy and this in information theory terms is called the KL divergence. It's one of many divergences, but it's a measure of the similarity of two distributions. This is a nice intuition here, by computing the maximum likelihood estimate of the parameters we're trying to minimize the distance between two distributions, more precisely between the data distribution given by the data set - the empirical data distribution and the distribution of our model given its parameters $\mathbf{w}$.

2.2 Computation graphs.

In the second unit of this lecture we will introduce computation graphs, which is a fundamental concept to understand larger changes of computations and also the computation of gradients in this large chains of computations. Unfortunately it's not true that these gradients are so simple to compute in general. For more complex models that we're interested in this lecture, in particular for deep neural networks, where you simply can't derive the gradients with respect to any of these gigantic number of parameters just using pen and paper. So how can we basically efficiently compute the gradients in the general?

2.2.1 Key idea.

The key idea of computation graphs is to **decompose** complex computations into sequences of very simple, more atomic assignments. We call this sequence of assignments a **computation graph** or **source code**. The **forward pass** takes a training point $(\mathbf{x}, y)$ as input and computes a loss, e.g.:

$$\mathcal{L} = -\log p_{model}(y | \mathbf{x}, \mathbf{w})$$

As we will see, gradients $\nabla_{\mathbf{w}} \mathcal{L}$ can be computed using a **backward pass**. Both, the forward pass and the backward pass are **efficient** due to the use of dynamic programming, i.e., storing and reusing intermediate results. This decomposition and reuse of computation is key to the success of the **backpropagation algorithm**, the primary workhorse of deep learning. It would be unthinkable to optimize deep networks with millions or billions of parameters without this decomposition and reuse of computation. That's why the backpropagation algorithm which adjusts these millions and billions of parameters in our deep models is still the primary workhorse for deep learning today.

2.2.2 Computation graph.

A computation graph has three kinds of nodes: input nodes in green, parameter nodes in orange and compute nodes in red. Also the loss function is a compute node in our setting. The input nodes don't have parameters they are the input of our learning problem, they are basically the dataset, this is where the data set goes into the computation graph. The parameter nodes is where the parameters of the model are stored and this is what we're interested in when backpropagating gradients. We need to backpropagate gradients for all nodes, but

what we're really interested in the end is the gradients for these parameters, because this is where the updates are applied to. Finally we have the compute nodes which could be over the final node (the loss function) or intermediate compute nodes that take inputs, for instance input nodes or previous compute nodes or parameters and produce a result.

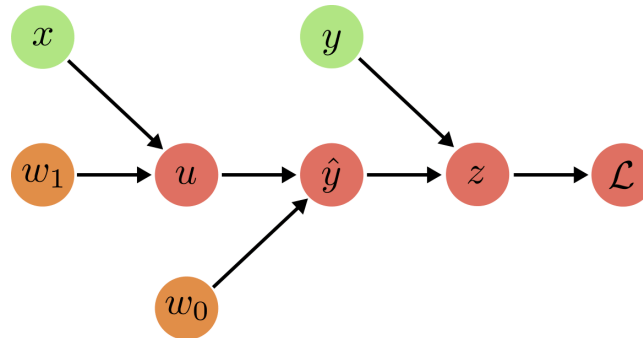


Figure 13: **Computation graph.** Example of the computation graph for linear regression.

We're looking at a very specific example of linear regression. This is the source code here:

- (1) $u = w_1 x$
- (2) $\hat{y} = w_0 + u$
- (3) $z = \hat{y} - y$
- (4) $\mathcal{L} = z^2$

This is how we write the linear regression problem, which we could also write in a single line, we write it in as a sequence of atomic operations. Now there is multiple levels of granularities that we can use. This is a very fine level of granularity, but we can use more coarse-grained levels of granularity as well and what we want to choose depends on how big we want to make these atomic units, which level of atomic units we still can handle. So in this case here we could combine (1) and (2), then we would change this computation graph.

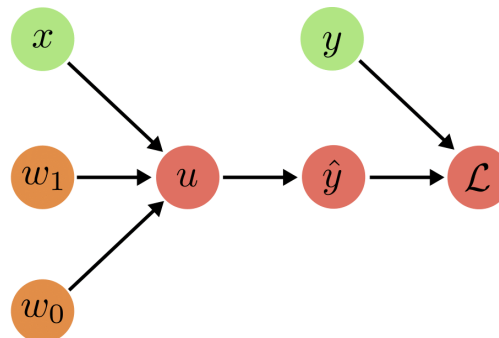


Figure 14: **Computation graph with another level of granularity.** Example of the computation graph with another level of granularity for linear regression.

Steps (1) and (2) have collapsed into one step, which directly computes this affine transformation.

We can also for instance collapse the loss, we can compute not these two things here independently but we don't want to compute them jointly:

So now we have only two steps:

- (1) $\hat{y} = w_0 + w_1 x$
- (2) $\mathcal{L} = (\hat{y} - y)^2$

Here is the example for the logistic regression:

- (1) $u = w_0 + w_1 x$
- (2) $\hat{y} = \sigma(u)$
- (3) $\mathcal{L} = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$

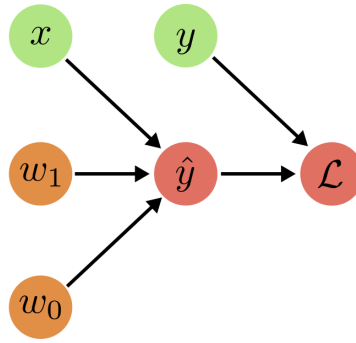


Figure 15: **Computation graph with another level of granularity.** Example of the computation graph with another level of granularity for linear regression.

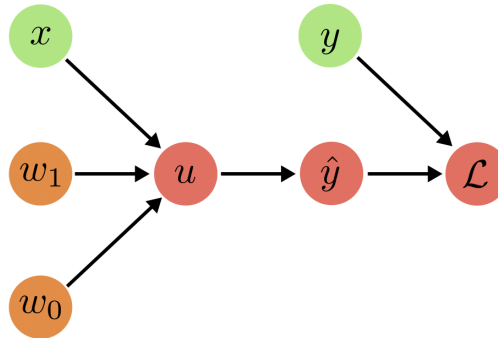


Figure 16: **Computation graph of logistic regression.** Example of the computation graph for logistic regression.

We could also write this in terms of vectors $\mathbf{w}$ and $\mathbf{x}$:

- (1) $u = \mathbf{w}^\top \mathbf{x}$
- (2) $\hat{y} = \sigma(u)$
- (3) $\mathcal{L} = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$

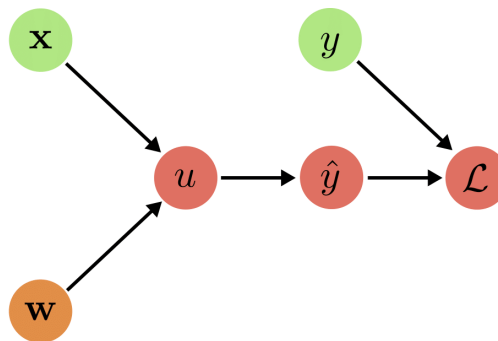


Figure 17: **Computation graph with vector representation.** Example of the computation graph with vector representation for logistic regression.

What we can also do is we can stack two of these operations behind each other:

- (1) $\mathbf{h} = \sigma(\mathbf{W}_1^\top \mathbf{x})$
- (2) $\hat{y} = \sigma(\mathbf{w}_2^\top \mathbf{h})$
- (3) $\mathcal{L} = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$

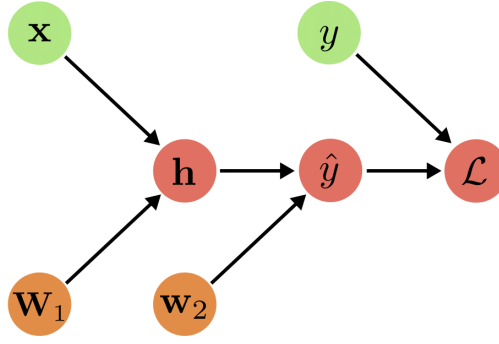


Figure 18: **Multi-Layer Perceptron.** Example of the computation graph for Multi-Layer Perceptron.

2.3 Backpropagation.

Our goal is to find gradients of negative log likelihood:

$$\nabla_{\mathbf{w}} \sum_{i=1}^N \underbrace{-\log p_{\text{model}}(y_i | \mathbf{x}_i, \mathbf{w})}_{\mathcal{L}(y_i, \mathbf{x}_i, \mathbf{w})} \quad (17)$$

or more generally of a loss function

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{y}, \mathbf{X}, \mathbf{w}) = \nabla_{\mathbf{w}} \sum_{i=1}^N \mathcal{L}(y_i, \mathbf{x}_i, \mathbf{w}) = \sum_{i=1}^N \nabla_{\mathbf{w}} \mathcal{L}(y_i, \mathbf{x}_i, \mathbf{w}) \quad (18)$$

given a dataset $\mathcal{X} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ with N elements. In the following, we consider the computation of gradients wrt. a single data point: $\nabla_{\mathbf{w}} \mathcal{L}(y_i, \mathbf{x}_i, \mathbf{w})$. The gradient with respect to the entire dataset $\mathcal{X}$ is obtained by summing up all individual gradients.

2.3.1 Chain rule.

In order to understand the backpropagation algorithm there's basically just one rule that we have to understand and that's the chain rule:

$$\frac{d}{dx} f(g(x)) = \frac{df}{dg} \frac{dg}{dx} \quad (19)$$

We're also going to need the so-called multivariate chain rule:

$$\frac{d}{dx} f(g_1(x), \dots, g_M(x)) = \sum_{i=1}^M \frac{\partial f}{\partial g_i} \frac{dg_i}{dx} \quad (20)$$

The multivariate chain rule is concerned with compositions of functions, where in the argument of a function we have a set of functions and each of these functions here depends on the same variable x .

2.3.2 Backpropagation algorithm.

Let's look at the most simple form of backpropagation algorithm, one of the most basic examples, which is basically just the chain rule in a slightly different form. For now we're going to not make any distinction between node types.

We're interested in the gradient of this loss with respect to the loss itself or the variable y or the variable x .

Forward Pass:

- (1) $y = y(x)$
- (2) $\mathcal{L} = \mathcal{L}(y)$

We're going to after running this forward pass, actually calculating the value of $\mathcal{L}$ for a particular input x , we are going to run a so called backward pass, that's where the name backpropagation is coming from. **Backward**

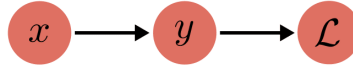


Figure 19: **Simple computation graph.**

Pass:

$$(2) \quad \frac{\partial \mathcal{L}}{\partial y} = \frac{\partial \mathcal{L}}{\partial \mathcal{L}} \frac{\partial \mathcal{L}}{\partial y} = \frac{\partial \mathcal{L}}{\partial y}$$

$$(1) \quad \frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} \frac{\partial y}{\partial x}$$

We will indicate forward pass with the black arrows and backward pass with the blue. We backpropagate gradients from the output node, which is always $\mathcal{L}$ in our case, to each individual node, such that at each individual node we can read off the gradients with respect to each of these nodes. We're interested in gradients of $\mathcal{L}$ with respect to any variable in this computation graph and we're going to use two different colors here for indicating two different types of quantities. Red color for backpropagated gradients and blue color for local gradients, that are locally computed based on this assignment formulas (1) and (2).

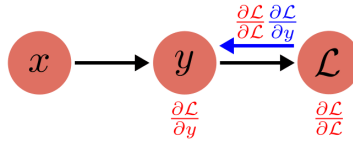


Figure 20: **Simple computation graph with backprop.**

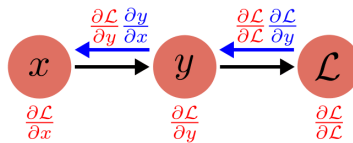


Figure 21: **Simple computation graph with backprop, step 2.**

The backward pass starts at the output node $\mathcal{L}$ and it computes gradients backwards. The first thing it does it computes the gradient of $\mathcal{L}$ with respect to itself which is obviously 1. In order to compute the gradient of $\mathcal{L}$ with respect to y we need to apply the chain rule. We need to compute the gradient of $\mathcal{L}$ with respect to the gradient of $\mathcal{L}$ times the gradient of $\mathcal{L}$ with respect to the gradient of y . Now we can do one step further, now

we're calculating the first expression the gradient with respect to the first variable, which we have in the forward pass computed first. Now we compute the gradient of $\mathcal{L}$ with respect to x , this is exactly what we wanted to compute. Again the chain rule, it's the gradient of $\mathcal{L}$ with respect to y times the gradient of y with respect to x . What we're ultimately interested in is to backpropagate gradients of course to the weights of a parametric function such as logistic regressor or a neural network. For all the values at a particular iteration we know the values, so we will we'll always insert the actual gradients and not the symbolic gradients.

Let's look at a slightly more difficult example. This is an example where we have a fan out bigger than one situation. Fan out means from a particular node there are multiple outgoing connections. In case y has multiple connections.

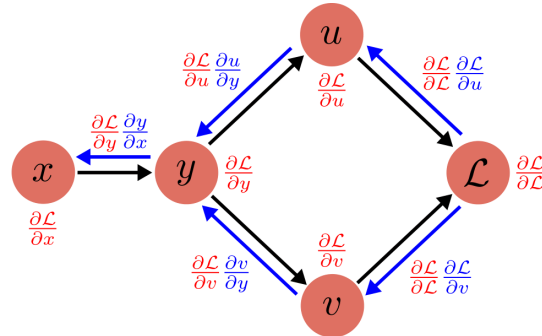


Figure 22: **Backpropagation: Fan-Out.**

Forward pass looks as follows:

- (1) $y = y(x)$
- (2) $u = u(y)$
- (2) $v = v(y)$
- (3) $\mathcal{L} = \mathcal{L}(u, v)$

Lets compute the backward pass now:

$$\begin{aligned}
 (3) \quad \frac{\partial \mathcal{L}}{\partial u} &= \frac{\partial \mathcal{L}}{\partial \mathcal{L}} \frac{\partial \mathcal{L}}{\partial u} = \frac{\partial \mathcal{L}}{\partial u} \\
 (3) \quad \frac{\partial \mathcal{L}}{\partial v} &= \frac{\partial \mathcal{L}}{\partial \mathcal{L}} \frac{\partial \mathcal{L}}{\partial v} = \frac{\partial \mathcal{L}}{\partial v} \\
 (2) \quad \frac{\partial \mathcal{L}}{\partial y} &= \frac{\partial \mathcal{L}}{\partial u} \frac{\partial u}{\partial y} + \frac{\partial \mathcal{L}}{\partial v} \frac{\partial v}{\partial y} \\
 (1) \quad \frac{\partial \mathcal{L}}{\partial x} &= \frac{\partial \mathcal{L}}{\partial y} \frac{\partial y}{\partial x}
 \end{aligned}$$

It's important to remember whenever we have such a fan out situation, whenever one variable sends to multiple variables we need to in the back propagation path is to sum up all the gradients and this is what has happened here.

2.3.3 Implementation.

Now we are basically ready to implement the backpropagation algorithm. A convenient way to do so is to consider each variable or node as an object that has certain attributes, for instance the value ($x.value$) and the gradient ($x.grad$). Values are computed in the forward pass. Here is the sequence of assignments in python:

```

x.value = Input
y.value = y(x.value)
u.value = u(y.value)
v.value = v(y.value)
L.value = L(u.value, v.value)

```

The value of x is an Input, value of y is function y evaluated at x value and then u value is the function u evaluated at y value and then v is also evaluated at y value. And then finally we have a Loss function.

Similarly we can look at the backward pass. First of all we would set the gradients to zero, because we'll gonna additively update.

```

x.grad = y.grad = u.grad = v.grad = 0
L.grad = 1
u.grad += L.grad * (∂L/∂u)(u.value, v.value)
v.grad += L.grad * (∂L/∂v)(u.value, v.value)
y.grad += u.grad * (∂u/∂y)(y.value)
v.grad += v.grad * (∂v/∂y)(y.value)
x.grad += y.grad * (∂y/∂x)(x.value)

```

In order to evaluate gradient we need the values that have been computed in the forward pass. The gradient itself is a function.

Let's look at a slightly more real example in the sense of machine learning. This is a logistic regression example. BCE denotes Binary Cross Entropy Loss.

Forward Pass:

$$\begin{aligned}
 (1) \quad & u = w_0 + w_1 x \\
 (2) \quad & \hat{y} = \sigma(u) \\
 (3) \quad & \mathcal{L} = \underbrace{-y \log \hat{y} - (1 - y) \log(1 - \hat{y})}_{\text{BCE}(\hat{y}, y)}
 \end{aligned}$$

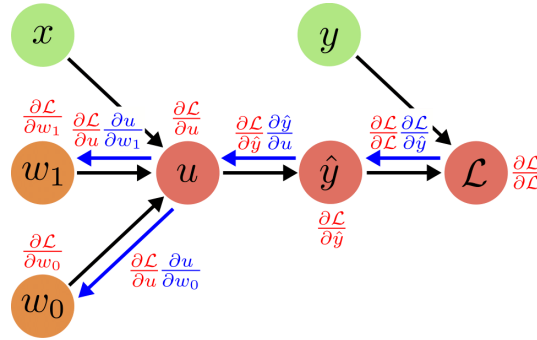


Figure 23: **Backpropagation: Log regression**

We want to calculate the gradients, but we're not interested in the gradients with respect to any variable in this graph, for instance we're not interested in the gradients with respect to these green input variables here, we could calculate them and sometimes we really do, like in the case of style transfer. That's the beauty about the back propagation algorithm that it can compute the gradients with respect to any variable can even compute the second derivative by executing it twice. But in this case here when we're interested in training the parameters of the logistic regression model. We're really just interested in back propagating the gradients from $\mathcal{L}$ to w_1 and w_0 so that we obtain the gradients of $\mathcal{L}$ with respect to w_1 and w_0 . Which are then used in the gradient based optimization in an iteration loop in order to update the gradient the parameters of the model. Let's apply the back propagation algorithm to this example:

$$\begin{aligned}
 (3) \quad & \frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{\partial \mathcal{L}}{\partial \mathcal{L}} \frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \\
 (2) \quad & \frac{\partial \mathcal{L}}{\partial u} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial u} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \sigma(u)(1 - \sigma(u)) \\
 (1) \quad & \frac{\partial \mathcal{L}}{\partial w_0} = \frac{\partial \mathcal{L}}{\partial u} \frac{\partial u}{\partial w_0} = \frac{\partial \mathcal{L}}{\partial u} \\
 (1) \quad & \frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial u} \frac{\partial u}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial u} x
 \end{aligned}$$

2.3.4 Summary

We can write mathematical expressions as a computation graph, that decomposes complex expressions into much simpler computations that are tractable and where we can apply efficient dynamic programming to solve

for both - the values in the forward pass and the gradients in the backward pass.

The values are efficiently computed in the forward pass and the gradients are computed in the backward pass. Multiple incoming gradients are summed up (multivariate chain rule).

Modularity: Each node must only "know" how to compute gradients wrt. its own arguments.

One fw/bw pass per data point:

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{y}, \mathbf{X}, \mathbf{w}) = \sum_{i=1}^N \underbrace{\nabla_{\mathbf{w}} \mathcal{L}(y_i, \mathbf{x}_i, \mathbf{w})}_{\text{Backpropagation}} \quad (21)$$

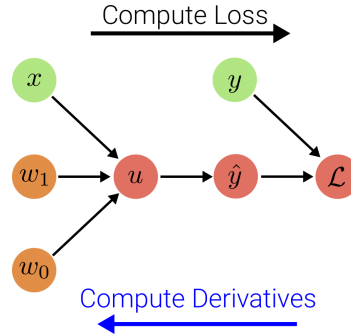


Figure 24: **Backpropagation: forward-backward**

2.4 Educational Framework.

Now we're going to introduce the educational framework, which is 160 lines of Python-NumPy code that implement a deep learning framework. It has been established in the context of a class taught by David McAllester, one of the pioneers of artificial intelligence at TTI Chicago. This educational framework allows us to understand the inner workings of a deep learning framework in depth. When listening to a lecture or a talk it is often easy to miss the details that are required to get a good understanding of how the discussed concepts actually work and only by implementing these concepts oneself one gets a good understanding of what has actually been missed. The other reason why we're using this educational framework is that it's very compact it allows us to understand each single line, it is a little bit less than 150 lines of Python-NumPy code. Let's look at this educational framework a little bit in more detail. It's using python, which means that variables are bound to objects. So for instance we have variables that describe the input or the labels or that are parents simply of particular compute nodes in that computation graph or we have values, which are attributes of these variables. The different types of nodes in the computation graph are implemented as classes. We have input classes parameter classes and computational node classes. So for instance from the compute node we can inherit a class that's called sigmoid and that implements the functionality of the sigmoid non-linearity within a logistic regression model or deep neural network.

```

class Input:
    def __init__(self):
        pass

    def addgrad(self, delta):
        pass

class Parameter:
    def __init__(self, value):
        self.value = DT(value)
        Parameters.append(self)

    def addgrad(self, delta):
        self.grad += np.sum(delta, axis = 0)

    def UpdateParameters(self):
        self.value -= learning_rate*self.grad

class CompNode:
    def addgrad(self, delta):
        self.grad += delta

```

Here we can see the abstract definition of the base classes. The input class has an initialization function and a function for adding the gradients, which are empty, because we are not interested in adding gradients to the input variables and also because input variables don't depend on any other variables. The parameter class has a initialization function that copies and stores the value of that parameter in the class itself and appends these parameters to a parameter vector, because later on we'll need to sum up these parameters in order to get the computation result. It also has a function for adding up gradients, this function simply sums up all the gradients along the data dimension. This model computes the values in the forward pass and the gradients in the backward pass for all the data points simultaneously and this is for efficiency reasons, because it allows to utilize efficient matrix operations in python. The Parameter class also has an UpdateParameter function that takes a step into the gradient direction, this is what is utilized during gradient descent. The compute node function also has a function for adding gradients, because we also need to update the gradients of the compute nodes themselves. In order to execute the computation graph we need to define a forward and a backward function.

```

def Forward():
    for c in CompNodes: c.forward()

def Backward(loss):
    for c in CompNodes + Parameters:
        c.grad = np.zeros(c.value.shape, dtype = DT)
        loss.grad = np.ones(loss.value.shape)/len(loss.value)
    for c in CompNodes[::-1]:
        c.backward()

def UpdateParameters():
    for p in Parameters: p.UpdateParameters()

```

Forward function takes the inputs x and y and the current state of the parameters w and goes left to right through the computation graph in order to compute all the intermediate values all the way until the head node $\mathcal{L}$. We have stored all these computation nodes in a list `CompNodes` and we have stored them in a way such that they are stored left to right, which means that when we arrive at any particular node in the sorted list, then we know that all the previous nodes have already been computed, so we can utilize their values. Thus in the forward pass of the backpropagation algorithm we simply go through the sorted list of computation nodes and for each computation node apply the forward function that is implemented inside that computation node. For the backward pass we first set all the gradients of all the computation nodes and all the parameters to zero and then we go backwards. We go backwards starting from $\mathcal{L}$ and propagate the gradients backward iteratively calling the backward function of each compute node in order to send gradient updates to its parents. Then finally we have an UpdateParameters function that loops through a list of parameters and calls the UpdateParameter function for each parameter.

Remark: Forward() and Backward() functions compute the forward and backward passes respectively over the

entire data set and the reason for this is that matrix operations are very cheap, because they are implemented very efficiently in python, while if we would go over the data set with loops we would be much slower. This vectorization is very important to exploit, furthermore if we would have GPUs hardware available, then we could even parallelize this computation, because the forward pass of each individual data point is independent of each other data point and the backward pass of each individual data point is independent of the backward pass of our data points.

Lets look at the concrete example of the computation node of sigmoid function:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

```
class Sigmoid(CompNode):
    def __init__(self, x):
        CompNodes.append(self)
        self.x = x

    def forward(self):
        bounded = np.maximum(-10, np.minimum(10, self.x.value))
        self.value = 1 / (1 + np.exp(-bounded))

    def backward(self):
        self.x.addgrad(self.grad * self.value * (1 - self.value))
```

Here we can see the python definition of the sigmoid class, which inherits from the class computation node. It has three functions: an initialization function, a forward function and a backward propagation function. At initialization time we simply add the node itself to the list of computation nodes and we store the parent of that node in that class itself. *self.x = x* means take the parent which is an input that could be another computation node let's say an affine computation node that does some computation and then inputs to the sigmoid function. In the forward function we calculate the sigmoid expression. So first we compute a bounded value of the value itself in order to avoid numerical problems and then we implement the sigmoid function. For the backward pass we implement derivative of sigmoid function, but what we do actually is we implement that function and multiply that function with the back propagated gradient, the gradient at the node itself and pass this as a message further on to the parent.

Remark: It's important to note that in this backward pass the gradient is sent to the parent node *self.x*. Let's execute a concrete minimal example.

Execution Example:

- Load data **X** and labels **y**
- Initialize parameters $\mathbf{w}^0$
- Define computation graph
- For all iterations do
 - Forward Pass
 $\mathcal{L}(\hat{y}_i = f_{\mathbf{w}}(\mathbf{x}_i), y_i)$
 - Backward Pass
 $\nabla_{\mathbf{w}} \mathcal{L}(\hat{y}_i, y_i)$
 - Gradient Update
 $\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \sum_{i=1}^N \nabla_{\mathbf{w}} \mathcal{L}(\hat{y}_i, y_i)$

```
import edf

# data loading
edf.clear_compgraph()
x = edf.Input()
y = edf.Input()
x.value = Load(data)
y.value = Load(labels)

# initialization of parameters
params_1 = edf.AffineParams(nInputs, nHiddens)
params_2 = edf.AffineParams(nHiddens, nLabels)

# definition of computation graph
h = edf.Sigmoid(edf.Affine(params_1, x))
p = edf.Softmax(edf.Affine(params_2, h))
L = edf.CrossEntropyLoss(p, y)

# gradient descent
for i in range(iterations):
    edf.Forward()
    edf.Backward(L)
    edf.UpdateParameters()
```